一.初识Express

1. Express 简介

官方给出的概念:Express是基于Node.js平台,快速、开放、极简的**Web开发框架**。

通俗的理解:Express的作用和Node.js内置的http模块类似,是专门用来创建Web服务器的。

- **Express的本质**:就是一个npm上的**第三方包**,提供了**快速创建服务器**的便捷方法。
- 创建Web服务器的两种方式
    - Express
    - Node.js的内置模块——http
        - 既然有了内置模块http搭建web服务器，那为什么还要用Express？ http内置模块用起来很复杂,开发效率低;**Express是基于内置的http模块进一步封装出来的**,能够极大的提高开发效率。
- Express的用途：方便、快速的创建Web网站的服务器或API接口的服务器。
    - 前端最见的两种服务器
        - **Web 网站服务器**:专门对外提供Web网页资源的服务器。
        - **API 接口服务器**:专门对外提供API 接口的服务器。

2.Express的基本使用

```
npm i express
或
npm i express@xxx
```

2.1创建基本的Web服务器

```
// 1. 导入 express
const express = require('express')

// 2. 创建 web 服务器
const app = express()

// 3. 调用 app.listen(端口号,启动成功后的回调函数),启动服务器
app.listen(80,()=>{
    console.log('express server running at http://127.0.0.1')
})
```

2.2 监听GET请求

通过**app.get()方法,可以监听客户端的GET请求**,具体的语法格式如下:

```
// 参数1:客户端请求的 URL 地址
// 参数2:请求对应的处理函数
//       req:请求对象(包含了与请求相关的属性与方法)
//       res:响应对象(包含了与响应相关的属性与方法)
app.get('请求URL',function(req,res){/*处理函数*/})
```

2.3 监听POST请求

通过**app.post()方法,可以监听客户端的POST请求**,具体的语法格式如下:

```
app.post('请求URL',function(req,res){/*处理函数*/})
```

2.4 把内容响应给客户端

通过**res.send()**方法,可以把处理好的内容,发送给客户端:

```
app.get('/user', (req, res) =>{
    // 向客户端发送 JSON 对象
    res.send({ name: 'zs', age: 20, gender:'男'})
})

app.post('/user', (req, res) => {
    // 向客户端发送文本内容
    res.send('请求成功')
})
```

> res 响应对象
>
> 在服务器的request事件处理函数中,如果**想访问与服务器相关的数据或属性**,可以使用如下的方式:
>
> ```
> server.on('request', (req, res) => {
>  // res 是响应对象,它包含了与服务器相关的数据和属性,例如:
>  // 要发送到客户端的字符串，客户端呈现的内容 ⭐⭐⭐⭐⭐
>  const str = 'Your request url is ${req.url}, and request method is ${req.method}'
>  // res.end()方法的作用:
>  //向客户端发送指定的内容,并结束这次请求的处理过程。即用户端能看到的内容 ⭐⭐⭐⭐⭐
>  res.end(str)
> })
> ```
>
> req 请求对象

2.5 获取URL中携带的查询参数

通过**req.query**对象,可以访问到客户端通过查询字符串的形式,发送到服务器的参数:

```
app.get('/', (req, res) => {
    // req.query 默认是一个空对象
    //客户端使用 ?name=zs&age=20 这种查询字符串形式,发送到服务器的参数,
    // 可以通过 req.query 对象访问到,例如:
    // req.query.name   req.query.age
    console.log(req.query)
})
```

2.6 获取URL中的动态参数

通过**req.params**对象,可以访问到URL中,通过 : 匹配到的动态参数:

```
// URL 地址中,可以通过:参数名 的形式,匹配动态参数值
app.get('/user/:id', (req, res) => {
    // req.params 默认是一个空对象
    // 里面存放着通过:动态匹配到的参数值
    console. log(req.params)
})
```

注意:

- 只有 : 是固定写法，冒号后面的是自定义的参数名
- 可以有多个动态参数

```
app.get('/user/:id/:age', (req, res) => {
    // req.params 默认是一个空对象
    // 里面存放着通过:动态匹配到的参数值
    console. log(req.params)
})
```

GET http://127.0.0.1/user/1 返回:

```
{
    "id": "1"
}
```

3.托管静态资源

3.1 express.static()

express提供了一个非常好用的函数,叫做**express.static()**,通过它,我们可以非常方便地创建一个静态资源服务器,例如,通过如下代码就可以将public目录下的图片、CSS文件、JavaScript文件对外开放访问了:

```
app.use(express. static('public'))  // 将某一指定目录下的资源对外开放
```

现在,你就可以访问public目录中的所有文件了:

```
http://localhost:3000/images/bg.jpg
http://localhost:3000/css/style.css
http://localhost:3000/is/login.js
```

注意:Express在**指定的**静态目录中查找文件,并对外提供资源的访问路径。 因此,**存放静态文件的目录名不会出现在URL中**。

3.2 托管多个静态资源目录

多次调用 `express.static()` 即可:

```
app.use(express.static('./public'))
app.use(express. static('./files'))
```

注意:访问静态资源文件时,express.static()函数**会根据目录的添加顺序查找所需的文件。**

谁写在前面,就先访问谁,找到了资源直接返回,如果没找到,则看下一个静态资源目录

./ 表示同一级; ../表示上一层

3.3 挂载路径前缀

如果希望在托管的**静态资源访问路径**之前,**挂载路径前缀**,则可以使用如下的方式:

```
app.use('/public', express.static('public'))
```

现在,你就可以通过带有/public前缀地址来访问public目录中的文件了:

```
http://localhost:3000/public/images/kitten.jpg
http://localhost:3000/public/css/style.css
http://localhost:3000/public/js/app.js
```

这个前缀建议与目录名保持一致,也可以设置为其它的,例如:

`app.use('/abjs', express.static('public'))` `http://localhost:3000/abjs/js/app.js`

4. nodemon

作用：在编写调试 Node.js项目的时候,如果修改了项目的代码,则需要频繁的手动close掉,然后再重新启动,非常繁琐。现在,我们可以使用nodemon(https://www.npmjs.com/package/nodemon)这个工具,它**能够监听项目文件 的变动,当代码被修改后,nodemon会自动帮我们重启项目**,极大方便了开发和调试。

```
npm i -g nodemon
```

启动项目：

```
nodemon 项目名.js
```

一般方式： node 项目名.js

二.Express 路由

1.什么是路由

广义上来讲，路由就是**映射关系**

- 在Express中,路由指的是客户端的请求与服务器处理函数之间的**映射关系。**
- Express中的路由分3部分组成,分别是请求的类型、请求的URL地址、处理函数,格式如下:

```
app.METHOD(PATH, HANDLER)
```

示例：

```
// 匹配 GET 请求,且请求 URL 为/
app.get('/', function (req, res) {
res. send('Hello World!')

// 匹配 POST 请求,且请求 URL 为/

app.post('/', function (req, res) {
res. send('Got a POST request')
```

- 路由的匹配过程

  每当一个请求到达服务器之后,需要先经过路由的匹配,只有匹配成功之后,才会调用对应的处理函数。 在匹配时,**会按照路由的顺序进行匹配**,如果请求类型和请求的URL同时匹配成功,则Express会将这次请求,转 交给对应的function函数进行处理。

  - 按照**定义的先后顺序**进行匹配
  - **请求类型**和**请求的URL**同时匹配成功,才会调用对应的处理函数

2.路由的使用

2.1 最简单的用法

在Express中使用路由**最简单的方式,就是把路由挂载到app上**,示例代码如下:

```
const express = require('express')
// 创建 Web 服务器,命名为 app
const app = express()

// 挂载路由
app.get('/', (req, res) => { res.send('Hello World.') })
app.post('/', (req, res) => { res.send('Post Request.') })

//启动 Web 服务器
app.listen(80, () => { console.log('server running at http://127.0.0.1') })
```

注意：**这种用法不推荐**，因为随着路由数量的增长，那么挂载到app上的路由就会越来越多

2.2 模块化的路由

为了方便对路由进行模块化的管理,Express不建议将路由直接挂载到app上,而是推荐**将路由抽离为单独的模块。**

将路由抽离为单独模块的步骤如下:

- 创建路由模块对应的.js文件
- 调用express.Router()函数**创建路由对象**
- 向路**由对象上挂载具体的路由**
- 使用 module.exports**向外共享路由对象**
- 使用app.use()函数**注册路由模块**

```
const express = require('express')  // 1. 导入 express
const router = express.Router()   // 2. 创建路由对象

router.get('/user/list',function(req,res){ //3. 挂载获取用户列表的路由
    res. send('Get user list.')

router.post('/user/add',function(req,res){ // 4. 挂载添加用户的路由
    res. send('Add new user.')
})

module.exports = router          // 5. 向外导出路由对象
```

注册路由模块：

```
// 1. 导入路由模块
const userRouter = require('./router/user.js')

// 2. 使用 app.use()注册路由模块
app.use(userRouter)
```

app.use()函数的作用，就是来**注册全局中间件**，目前已遇到的：app.use()注册路由模块、托管静态资源

2.3 为路由模块添加前缀

类似于托管静态资源时,为静态资源统一挂载访问前缀一样,路由模块添加前缀的方式也非常简单:

```
// 1. 导入路由模块
const userRouter = require('./router/user.js')

//2. 使用 app.use()注册路由模块,并添加统一的访问前缀 /api
app.use('/api', userRouter)  // http://127.0.0.1/api/user/list
```

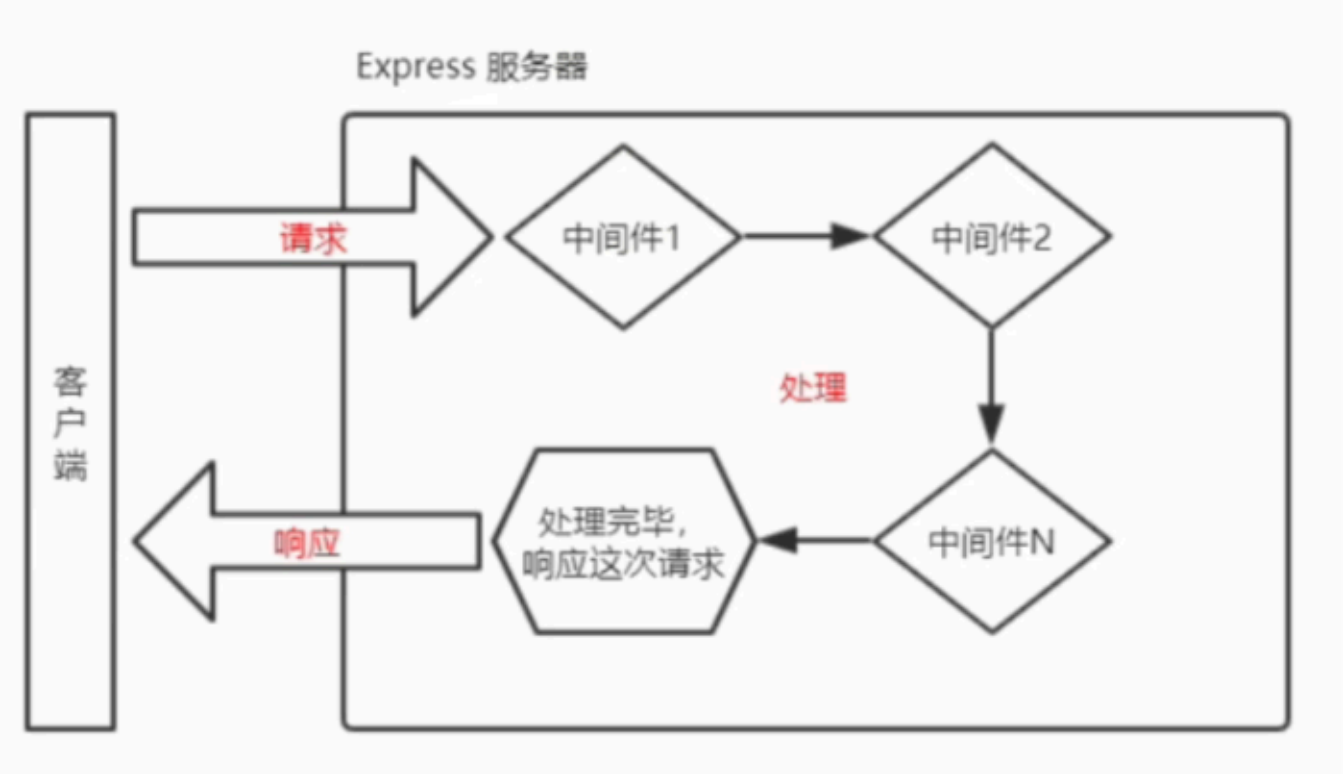
三.Express 中间件

1.什么是中间件

中间件(Middleware),特指**业务流程的中间处理环节**。 有输入与输出

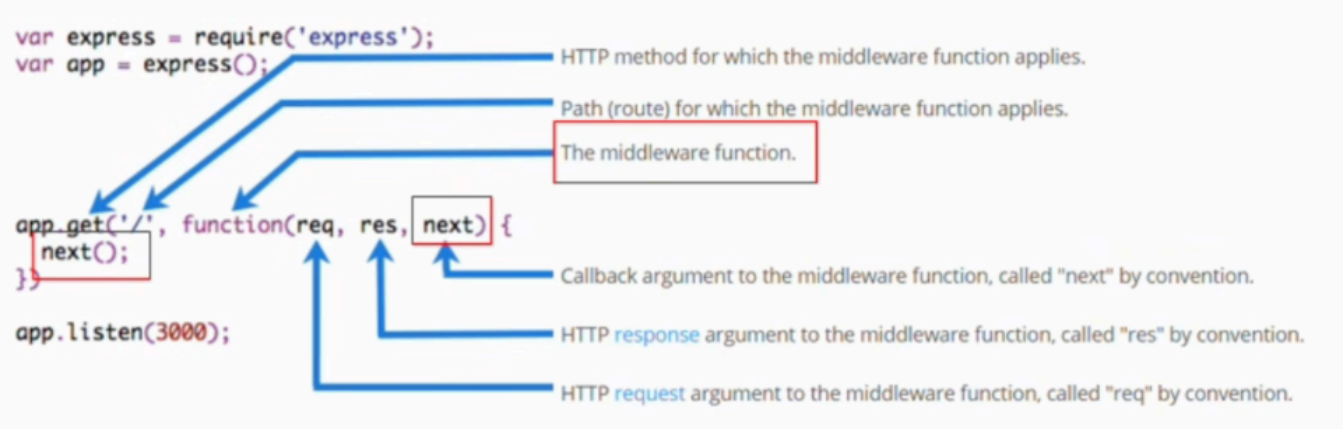
2.Express 中间件的调用流程

当一个请求到达Express的服务器之后,**可以连续调用多个中间件**,从而对这次请求进行预处理。



3. Express中间件的格式

Express的中间件,本质上就是一个function处理函数,Express中间件的格式如下:



注意:中间件函数的形参列表中,**必须包含next参数**。而路由处理函数中只包含req和res。

- next 函数的作用
  - next 函数**是实现多个中间件连续调用的关键**,它表示把流转关系转交给**下一个中间件或路由**。

4. Express中间件的体验

4.1 最简单的一个中间件函数

```
const express = require('express')
const app = express()

// 定义一个最简单的中间件函数
```

```
const middleware = (req,res,next)=>{
    console.log('中间件函数被调用了')

    // 注意:在当前中间件的业务处理完毕后,必须调用 next()函数
    next()  // 把流转给下一个中间件函数或路由处理函数
}

app.listen(80,()=>{
    console.log('express server running at http://127.0.0.1')
})
```

4.2全局生效的中间件

客户端发起的任何请求,到达服务器之后,都会触发的中间件,叫做**全局生效的中间件**。

通过调用**app.use(中间件函数)**,即可定义一个全局生效的中间件,示例代码如下:

```
// 常量mw 所指向的,就是一个中间件函数
const mw = function (req, res, next) {
    console.log('这是一个最简单的中间件函数')
    next()
}

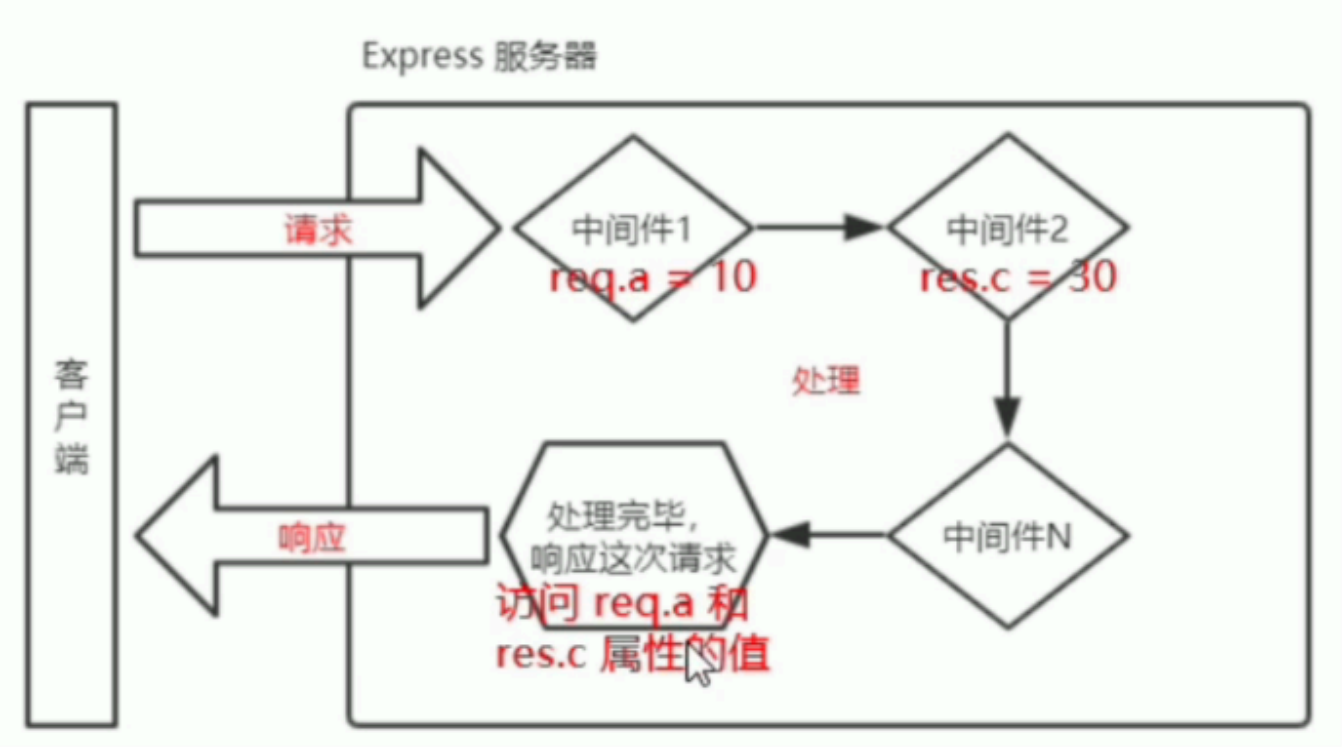
// 全局生效的中间件
app.use(mw)
```

简化形式:

```
// 全局生效的中间件
app.use(function (req, res, next) {
    console.log('这是一个最简单的中间件凶数')
    next()
})
```

4.3 中间件的作用

多个中间件之间,**共享同一份req和res**。基于这样的特性,我们**可以在上游的中间件中,统一为req或res对象添 加自定义的属性或方法,供下游的中间件或路由进行使用。**



个人感觉就是模块化的一种类型，把公共的部分封装起来(类比装饰器/语法糖)

```
const express = require('express')
const app = express()

// 定义一个最简单的中间件函数
const middleware = (req,res,next)=>{
    //获取到请求到达服务器的时间
    const time = Date.now()
    // 为req对象，挂载自定义属性，从而把时间共享给后面的所有路由
    req.startTime = time

    next()  // 把流转给下一个中间件函数或路由处理函数
}

// 将middleware挂载到app对象上,注册为全局生效的中间件
app.use(middleware)

app.get('/',(req,res)=>{
    res.send('Home Page' + req.startTime)
})

app.get('/user',(req,res)=>{
    res.send('User Page' + req.startTime)
})

app.listen(80,()=>{
    console.log('express server running at http://127.0.0.1')
})
```

4.4 定义多个全局中间件

可以使用app.use()连续定义多个全局中间件。**客户端请求到达服务器之后,会按照中间件定义的先后顺序依次进行调用**,示例代码如下:

```
const express = require('express')
const app = express()

// 定义第一个全局中间件
app.use((req,res,next)=>{
    console.log('调用了第一个全局中间件')
    next()
})

// 定义第二个全局中间件
app.use((req,res,next)=>{
    console.log('调用了第二个全局中间件')
    next()
})

// 定义路由处理函数
// 客户端请求到达服务器之后,会按照中间件定义的先后顺序依次进行调用
app.get('/',(req,res)=>{
    res.send('Home Page')
})

app.listen(80,()=>{
    console.log('express server running at http://127.0.0.1')
})
```

4.5 局部生效的中间件

不使用app.use()定义的中间件,叫做局部生效的中间件,示例代码如下:

```
const express = require('express')
const app = express()

// 1.定义中间件函数
const mw1 = (req,res,next)=>{
    console.log('调用了局部生效的中间件1')
    next()
}

// 2.定义路由处理函数  mw1 只会在该路由中生效
app.get('/',mw1,(req,res)=>{
    res.send('Home Page')
})

// 3.定义路由处理函数
app.get('/user',(req,res)=>{
    res.send('User Page')
})
```

```
app.listen(80,()=>{
    console.log('express server running at http://127.0.0.1')
})
```

4.6 定义多个局部中间件

可以在路由中,通过如下两种等价的方式,使用多个局部中间件:

```
//以下两种写法是“完全等价”的,可根据自己的喜好,选择任意一种方式进行使用
app.get('/', mw1, mw2, (req, res) => {res.send('Home page.') })
app.get('/', [mw1, mw2], (req, res) => {res.send('Home page.') })
```

按顺序调用

4.6 中间件的5个使用注意事项

- 被路由使用的中间件，一定要注册/定义在该路由之前
- 客户端发送过来的请求,可以**连续调用多个中间件**进行处理
- 执行完**中间件**的业务代码之后,不要忘记调用**next()函数**
- 为了防止代码逻辑混乱,调用next()函数后不要再写额外的代码
- 连续调用多个中间件时,多个中间件之间,共享req和res对象

5. 中间件的分类

- **应用级别**的中间件
  - 通过app.use()或app.get()或app.post(),**绑定到app实例上**的中间件,叫做应用级别的中间件
- **路由级别**的中间件
  - **绑定到express.Router()实例上**的中间件,叫做路由级别的中间件。它的用法和应用级别中间件没有任何区别。只不过,应用级别中间件是绑定到app实例上,路由级别中间件绑定到router实例上
- **错误级别**的中间件
  - 错误级别中间件的作用:专门**用来捕获整个项目中发生的异常错误**,从而防止项目异常崩溃的问题。
  - 格式:错误级别中间件的function处理函数中,必须有4个形参,形参顺序从前到后,分别是**(err,req,res,next)**。

```
app.get('/', function (req, res) {            // 1. 路由
  throw new Error('服务器内部发生了错误!')    // 1.1 抛出一个自定义的错误
  res.send('Home Page.')
})

app.use(function (err, req, res, next) {     // 2. 错误级别的中间件
  console.log('发生了错误: ' + err.message)   // 2.1 在服务器打印错误消息
  res.send('Error! ' + err.message)           // 2.2 向客户端响应错误相关的内容
})
```

1. **错误中间件的硬性规则**：必须四个参数 `(err, req, res, next)`，顺序不能变，Express 靠参数数量识别它是错误中间件。
2. 当路由里 `throw` 抛出错误，Express 会自动把错误交给这个错误中间件处理，不会继续执行后面的 `res.send('Home Page.')`（throw 之后代码直接终止）。
3. 注意：**这种捕获只对同步代码 throw 有效**，异步函数里 throw 是捕获不到的，异步错误要手动 `next(err)`。
4. ⚠️ 重点坑：错误中间件**位置必须放在所有路由之后**，放前面是不生效的。

- **Express内置**的中间件
  - 自Express 4.16.0版本开始,Express内置了3个常用的中间件
    - express.static 快速托管静态资源的内置中间件,例如:HTML文件、图片、CSS样式等(无兼容性)
    - express.json解析JSON格式的请求体数据(有兼容性,仅在4.16.0+版本中可用)
    - express.urlencoded 解析 URL-encoded格式的请求体数据(有兼容性,仅在4.16.0+版本中可用)
- **第三方**的中间件
  - 非Express 官方内置的,而是由第三方开发出来的中间件,叫做第三方中间件。在项目中,大家可以按需下载并配置第三方中间件,从而提高项目的开发效率。
    - 例如:在express@4.16.0之前的版本中,经常使用body-parser这个第三方中间件,来解析请求体数据。使用步骤如下:

      运行 npm install body-parser 安装中间件 使用 require 导入中间件 调用app.use()注册并使用中间件

四.使用Express写接口